to ensure its integrity and provide informative and timely feedback to users when the data they submit fails to meet your business rules.

One of the design goals behind the Razor Pages framework, as stated in the GitHub issue quoted in chapter 1, is to "simplify the code required to implement common page-focused patterns, e.g., dynamic pages, *CRUD forms*."

Razor Pages includes great features for minimizing the amount of code needed to create robust forms, process their data, and validate it. They are

- Form control tag helpers
- Model binding
- A validation framework

These topics leave us with a lot of ground to cover, so I focus on model binding and validation in this chapter and focus on tag helpers in the next chapter.

In chapter 3, you saw that query string values can be bound to handler method parameters through a feature called *model binding*. This feature also plays a key part in simplifying the processing of posted form values by automating the capture of data sent in an HTTP request and assigning it to C# code. You will explore model binding in detail in this chapter when you see how it works with `PageModel` properties and route data as well as handler parameters.

You will also see how to protect your application from bad data by validating user input both on the client and, crucially, on the server. Validation tag helpers and model binding combine to reduce the amount of code you need to write to protect your data integrity and inform users of errors in their submission.